

SPM: Message Passing Interface (MPI) (Part I)

Academic year 2025-2026

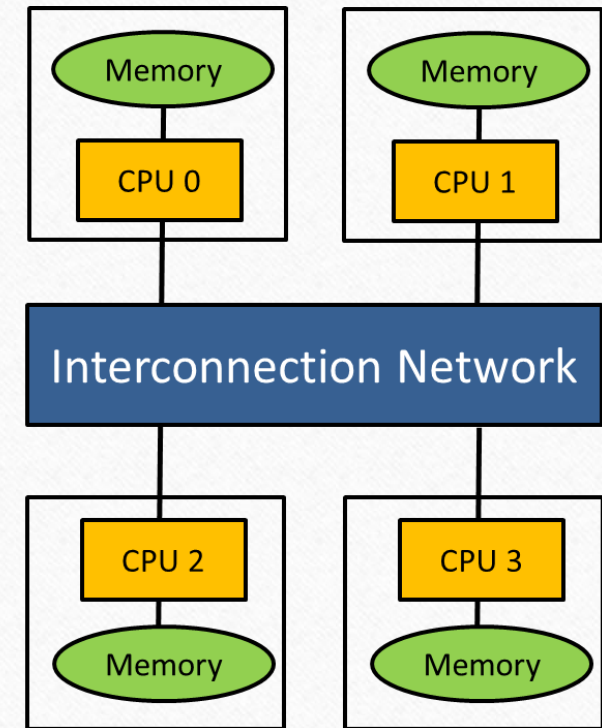
Prof. Massimo Torquati
massimo.torquati@unipi.it

Outline

- Introduction to MPI
- Execution model
- Communication taxonomy
 - Synchronous/asynchronous - Symmetric/asymmetric
- Communicators, ranks, tags
- Symmetric communication in MPI: synchronous vs. non-blocking
- Simple examples

Message Passing Model (Logical View)

- p processes, each with its **private address space**
- Data placement, partitioning, and movement are explicit responsibilities of the programme
- Most point-to-point interactions are two-sided:
 - one process that has the data sends it (e.g., MPI_Send)
 - one process that needs the data receives it (e.g., MPI_Recv)
- **Strengths:**
 - Relatively simple performance model
 - Offers high performance by co-locating data with computation
 - General model (portability) that can be used in all systems
- **Challenges:** explicit coordination can increase programming complexity



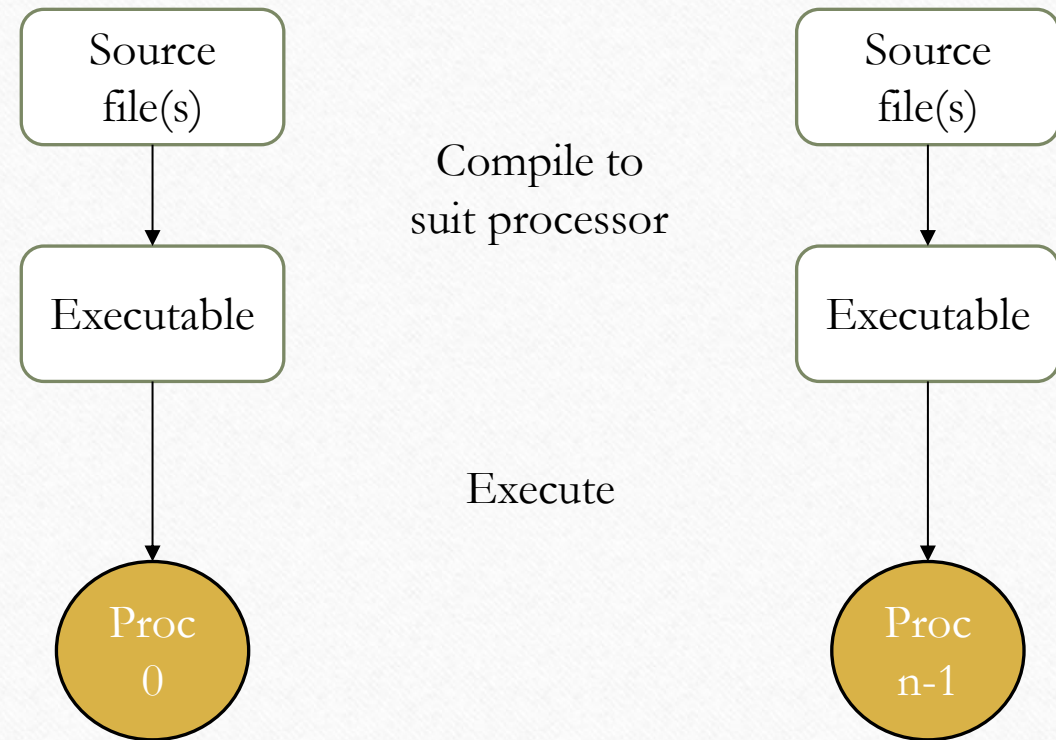
Distributed memory MIMD

User-level Message-Passing Programming

- Two primary mechanisms are needed:
 - A method of creating multiple processes, possibly mapped on different nodes
 - **MPMD**: Multiple-Program Multiple-Data, different executables in the same parallel applications
 - **SPMD**: Single-Program Multiple-Data, same executable, different behavior depending on process id (rank)
 - A method of sending and receiving messages
 - Send/receive operations have precise **completion semantics**: blocking and non-blocking
 - MPI also provides different send modes

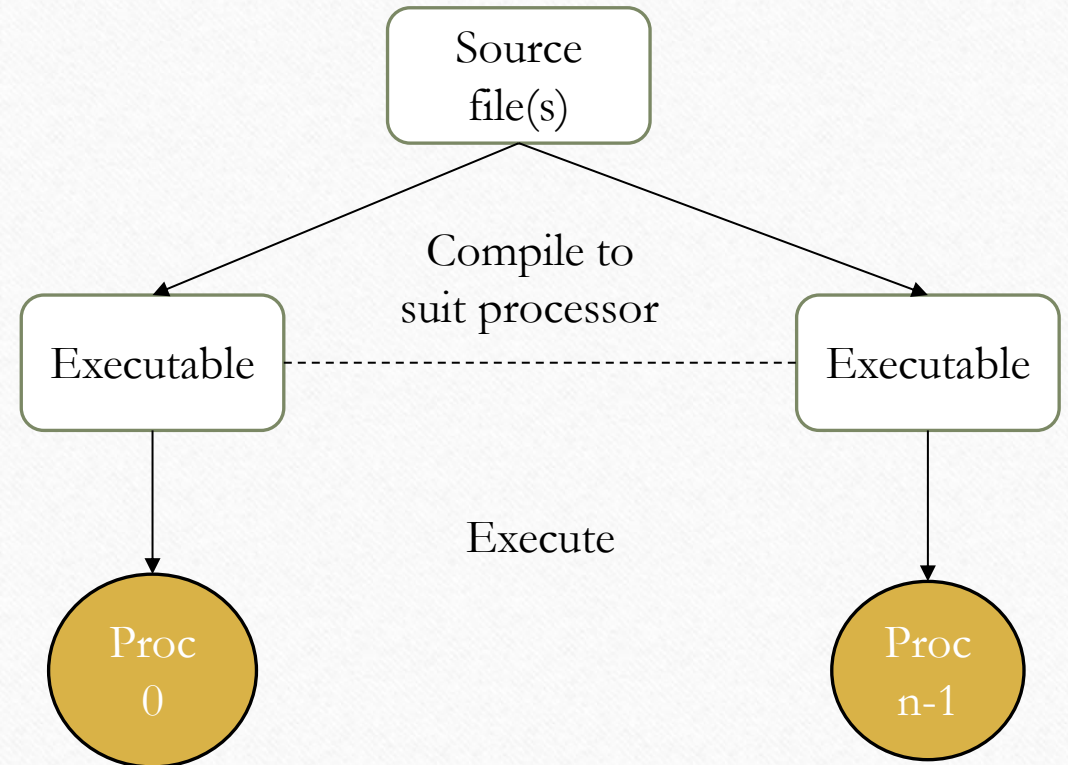
MPMD model

- Separate programs for each executable
- Different executables participate in the same parallel application.
- Processes cooperate through an explicit communication library, e.g., MPI
 - MPI example, one master and two Workers
 - `mpirun -n 1 Master : -n 2 Worker`



SPMD model

- The same executable is launched in multiple processes
 - Different logical flows merged into one program
 - Control statements select different parts for each process to execute
 - Each process has a unique rank and follows a different path depending on its rank
- SPMD is the most common MPI execution model
 - MPI also supports the MPMD applications



Message Passing in MPI

- MPI is a standard interface for message-passing programs
- MPI provides:
 - Process identification, i.e., ranks
 - Communication domains, i.e., communicators
 - Point-to-point communication
 - Collective communication
 - Synchronization and completion mechanisms
- MPI is not a language
 - Programs are written in C, Fortran, or other languages through bindings/wrappers
 - C++ bindings deprecated, in this course, we write C++ programs that use the C MPI interface
 - Unofficial/community bindings and wrappers exist for Python, Rust, Go, C#, ...

Message Passing in MPI

- No implicit shared address space (*). Communication and Synchronization operations are MPI library calls
 - Communication
 - Point-to-Point communication
 - Collective communication
 - Synchronization
 - Explicit synchronization, i.e., MPI_Barrier
 - Implicit synchronization induced by communication
 - Matching send/receive
 - Collective operations require all processes in the communicator to call them
 - Other functions for querying the MPI library runtime:
 - How many processes are in this communicator? (MPI_Comm_size)
 - What is my rank? (MPI_Comm_rank)
 - Has this non-blocking communication completed? (MPI_Test / MPI_Wait)
 - ...

(*) MPI-3 introduced shared-memory windows for processes mapped on the same node. They are explicit MPI objects, not ordinary shared variables.

Scope of the MPI Standard

- MPI is a large standard
- In this part we focus on
 - Process startup and finalization
 - Communicators and ranks
 - Point-to-point communication
 - Blocking and non-blocking send/receive
- Many other MPI features are available



- Point-to-point communication,
- Partitioned communication,
- Datatypes,
- Collective operations,
- Process groups,
- Communication contexts,
- Virtual Topologies for MPI Processes,
- Environmental management and inquiry,
- The Info object,
- Process initialization, creation, and management,
- One-sided communication,
- External interfaces,
- Parallel file I/O,
- Tool support,
- Language bindings for Fortran and C, and
- Additional topics in side-documents.

From MPI: A Message-Passing Interface Standard Version 4.1

MPI-4.1 was released in 2023 <https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report.pdf>

Core MPI functions

- Simple to get started. Simple programs use only six library routines
 - **MPI_Init**: initializes MPI
 - **MPI_Finalize**: terminates MPI
 - **MPI_Comm_size**: determines the number of processes in a communicator (e.g., **MPI_COMM_WORLD**)
 - **MPI_Comm_rank**: determines the rank of the calling process in the communicator
 - **MPI_Send**: sends a message (**blocking**)
 - **MPI_Recv**: receives a message (**blocking**)

Non-blocking send/receive variants, introduced later:

- **MPI_Isend**
- **MPI_Irecv**

SPMD model in MPI

- The same program is executed by all P processes (**SPMD model**)
- Each process chooses a different execution path depending on its rank
- The rank is unique within a communicator and range from 0 to P-1

```
...  
MPI_Init(&argc,&argv);           // no MPI call before this point  
int rank  
MPI_Comm_rank(MPI_COMM_WORLD, &rank);  
myFunction1();                   // executed by all processes  
switch(myID) {  
    case 0: foo1(); break;        // executed by process 0 only  
    case 1: foo2(); break;        // executed by process 1 only  
    default: foo3(); break;       // executed by all other processes  
}  
myFunction2();                   // executed by all processes  
MPI_Finalize();                  // no MPI call after this point  
...
```

Starting and Terminating MPI programs

- `int MPI_Init(int* argc, char*** argv)`
 - No MPI calls before this call
 - Parse the command line, *stripping off MPI arguments*
 - Initialize the MPI environment
- `int MPI_Finalize()`
 - No MPI calls after this call
 - Performs clean-up to terminate the MPI environment
- Return codes
 - `MPI_SUCCESS` on success
 - Otherwise, an `MPI_ERR_*` error code is returned
examples: `MPI_ERR_COMM`, `MPI_ERR_RANK`, `MPI_ERR_TAG`, `MPI_ERR_COUNT`, ...

Communicators

- **Communicator (MPI_Comm)**
 - Defines the scope of communications, i.e., the **communication domain**
 - A communicator consists of
 - a group of processes
 - a communication context
 - The processes in the group are allowed to communicate between each other
 - Ranks are local to a communicator
 - The same process may have different ranks in different communicators
 - Passed as an argument to all MPI message transfer routines
 - Processes can belong to multiple communication domains (i.e., domains can overlap)
 - Initially, all processes are enrolled in **MPI_COMM_WORLD**
 - Its group contains all application processes
 - Other communicators can be created dynamically for grouping processes
 - **MPI_Comm_split** / **MPI_Comm_create**

Communicator inquiry functions

- `int MPI_Comm_size(MPI_Comm comm, int* size)`
 - Stores in *size* the number of processes in the communicator *comm*
- `int MPI_Comm_rank(MPI_Comm comm, int* rank)`
 - Stores in *rank* the rank of the calling process in the communicator *comm*
 - If the communicator has size *P*, then $0 \leq rank < P$
 - Ranks are local to a communicator

Minimal «Hello World» in MPI

```
#include <stdio>
#include <mpi.h>
int main(int argc, char *argv[]) {
    MPI_Init(&argc, &argv);
    int size, rank;
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    std::printf("Hello from rank %d of %d\n", rank, size);
    MPI_Finalize();
}
```

Example: `mpi_helloworld.cpp`

MPI on the spmcluster

- In the `spmcluster` the MPI implementation installed is `Open MPI v5.0.3`
 - `ompi_info` to see configuration information
 - Open MPI v5 documentation: <https://docs.open-mpi.org/en/v5.0.x/>
- MPI directory: `/opt/ohpc/pub/mpi/`
 - `/opt/ohpc/pub` directory visible in all cluster nodes (NFS mounted directory)
- In general, on different SLURM partitions, the same MPI version can be configured in different ways (es. with CUDA or without CUDA support)
- SLURM and MPI:
 - `srun --mpi=pmix -N <nodes> -n <processes> ./myprog` // use the PMIx plugin (Process Management Interface for Exascale)
 - `salloc -N <nodes> -n <processes> --time 00:01:00` // allocating resources and then using `mpirun` to run processes on the nodes
`mpirun -n <processes> ./myprog`
- MPI applications can be compiled with: `mpicc/mpicxx`

MPI «Hello, world!»

- **Compilation:**

```
// compiling on the master node
mpicxx -Wall -O3 mpi_helloworld.cpp -o mpi_helloworld
// compiling on a cluster node using SLURM
srun -n 1 make mpi_helloworld
```

- **Execution:**

```
salloc -N 4 // suppose SLURM gives us node[01-04]
mpirun -n 4 ./mpi_helloworld // directly using mpirun

srun --mpi=pmix -n 4 ./hello_world // using SLURM
```

```
[torquati@spmln spmcode10]$ srun --mpi=pmix -N 8 -n 8 ./mpi_helloworld
Hello from rank 0 of 8 running on node01
Hello from rank 1 of 8 running on node02
Hello from rank 3 of 8 running on node04
Hello from rank 7 of 8 running on node08
Hello from rank 5 of 8 running on node06
Hello from rank 2 of 8 running on node03
Hello from rank 6 of 8 running on node07
Hello from rank 4 of 8 running on node05
[torquati@spmln spmcode10]$ salloc -N 8 -n 8 --time=00:01:00
salloc: Granted job allocation 569956
salloc: Nodes node[01-08] are ready for job
[torquati@spmln spmcode10]$ squeue --me
      JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
      569956    normal interact torquati  R        0:03       8 node[01-08]
[torquati@spmln spmcode10]$ mpirun -n 8 ./mpi_helloworld
Hello from rank 2 of 8 running on node03
Hello from rank 3 of 8 running on node04
Hello from rank 6 of 8 running on node07
Hello from rank 5 of 8 running on node06
Hello from rank 0 of 8 running on node01
Hello from rank 1 of 8 running on node02
Hello from rank 4 of 8 running on node05
Hello from rank 7 of 8 running on node08
[torquati@spmln spmcode10]$ scancel 569956
salloc: Job allocation 569956 has been revoked.
Hangup
```


Sending and Receiving Messages

- `int MPI_Send(const void* buf, int count, MPI_Datatype dt, int dest, int tag, MPI_Comm comm)`
- `int MPI_Recv (void* buf, int count, MPI_Datatype dt, int source, int tag, MPI_Comm comm, MPI_Status* status)`
- `source` and `dest` are the ranks of the processes in the communicator `comm`
- `count` is the number of elements of MPI datatype `dt`, not the number of bytes
 - The number of bytes is given by `count*type_size` where `type_size` can be obtained using `MPI_Type_size`
- The message `tag` is an integer value, $0 \leq \text{tag} < \text{MPI_TAG_UB}$
- A receive matches a message by `communicator`, `source`, and `tag`

Sending and Receiving Messages (cont.)

- Instead of specifying an exact source and tag, the receiver may use wildcards
 - `MPI_ANY_SOURCE`, matches a message from any process in `comm`
 - `MPI_ANY_TAG`, matches a message with any tag
- Wildcards can be used only on the receiver side
- If wildcards are used, the actual source and tag can be inspected through the `MPI_Status`
 - `status.MPI_SOURCE` and `status.MPI_TAG`
- Receiver-side constraints
 - **No partial reads.** One `MPI_Recv` matches one complete message
 - Receiving fewer than `count` elements is allowed
 - If the incoming message does not fit in the receiving buffer, `MPI_ERR_TRUNCATE` is raised

Receiver status

- The `MPI_Status` status, is an output object filled by receive and completion operations
 - For completed receive, it contains the source (`MPI_SOURCE`), tag (`MPI_TAG`), and error (`MPI_ERROR`) of the communication. Useful when `MPI_ANY_SOURCE` or `MPI_ANY_TAG` are used
 - It is possible to use `MPI_STATUS_IGNORE` if no information is needed
- `int MPI_Get_count(const MPI_Status* status, MPI_Datatype dt, int* count)`
 - Stores in `count` of how many elements of a given MPI datatype (`dt`) have been received
 - Useful when the receive buffer is larger than the incoming message
 - `MPI_Recv` may complete *even if fewer than the requested elements were received*

Some Primitive MPI Datatypes

- In MPI a message is described by the triple
 <buffer, count, datatype>
- The datatype tells the runtime how to interpret the memory pointed to by the buffer. The count parameter tells how many elements of that datatype are sent or received
- For contiguous arrays of primitive C/C++ values, **predefined MPI datatypes are enough**. For structs, matrices stored by columns, halos, subarrays, or non-contiguous memory regions, **MPI provides derived datatypes**
- Custom data types can be constructed by using specific MPI functions

MPI datatype	C/C++ type
MPI_CHAR	char
MPI_INT	int
MPI_LONG	long
MPI_LONG_LONG_INT	long long int
MPI_UNSIGNED	unsigned int
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_LONG_DOUBLE	long double
MPI_BYTE	raw byte

MPI_BYTE is for raw uninterpreted bytes, not for portable structured data

A simple program

Example 3.1. A simple 'hello world' example usage of point-to-point communication.

```
#include "mpi.h"
int main(int argc, char *argv[])
{
    char message[20];
    int myrank;
    MPI_Status status;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &myrank);
    if (myrank == 0) /* code for process zero */
    {
        strcpy(message, "Hello, there");
        MPI_Send(message, strlen(message)+1, MPI_CHAR, 1, 99,
                  MPI_COMM_WORLD);
    }
    else if (myrank == 1) /* code for process one */
    {
        MPI_Recv(message, 20, MPI_CHAR, 0, 99, MPI_COMM_WORLD, &status);
        printf("received :%s:\n", message);
    }
    MPI_Finalize();
    return 0;
}
```

Picture from MPI: A Message-Passing Interface Standard Version 4.1

Example: mpi_sendrecv.cpp

MPI Tags

- A tag is an integer label attached to a message
- Tags are used to distinguish different logical message kinds
 - e.g., data, control messages, termination
- A receive matches messages using communicator, source, and tag
- If a received message does not match the requested tag, it remains pending and may be matched by a later `MPI_Recv`

```
const int DATA_TAG = 0;  
const int EOS_TAG = 1;
```

```
MPI_Send(buffer, count, MPI_INT, dest, DATA_TAG, MPI_COMM_WORLD);  
MPI_Send(nullptr, 0, MPI_BYTE, dest, EOS_TAG, MPI_COMM_WORLD);
```


Communication Taxonomy

- Communication patterns
 - Point-to-point communication (one sender, one receiver)
 - Collective communication (all processes in a communicator participate)
 - Some collectives also perform data processing, e.g., `MPI_Reduce`
- Synchronization with the communication peer
 - Synchronous communication : rendezvous semantics
 - Asynchronous communication : the sender may complete without waiting for the receiver to start the matching receive
- Completion semantics
 - Blocking : the call returns only after the local operation is complete
 - The message buffer can be immediately reused
 - Non-blocking : the call starts the operation and returns immediately
 - Completion must be checked before accessing the message buffer

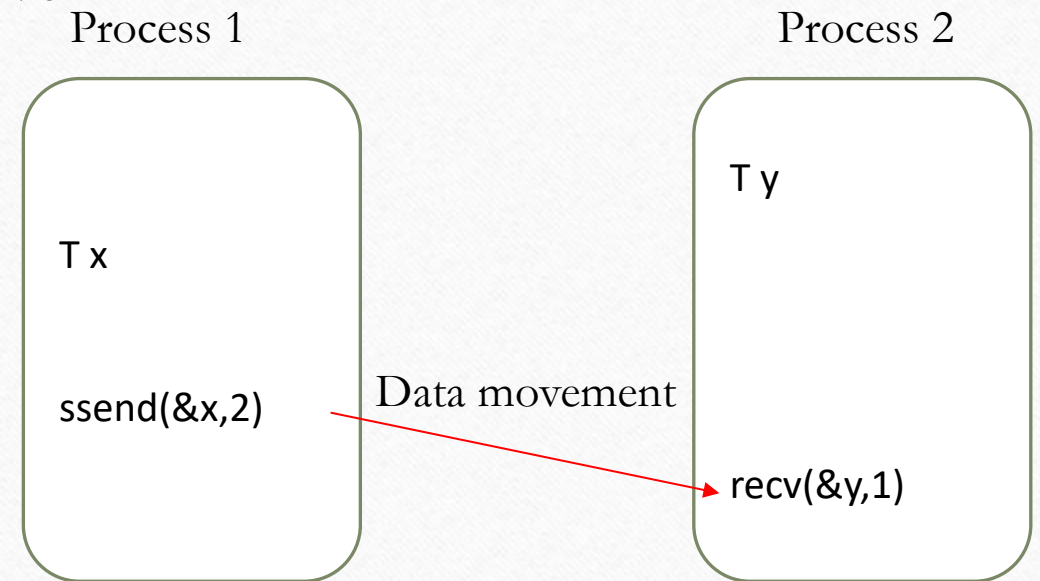
Synchronous $\neq$ Blocking
Asynchronous $\neq$ Non-blocking

MPI examples:

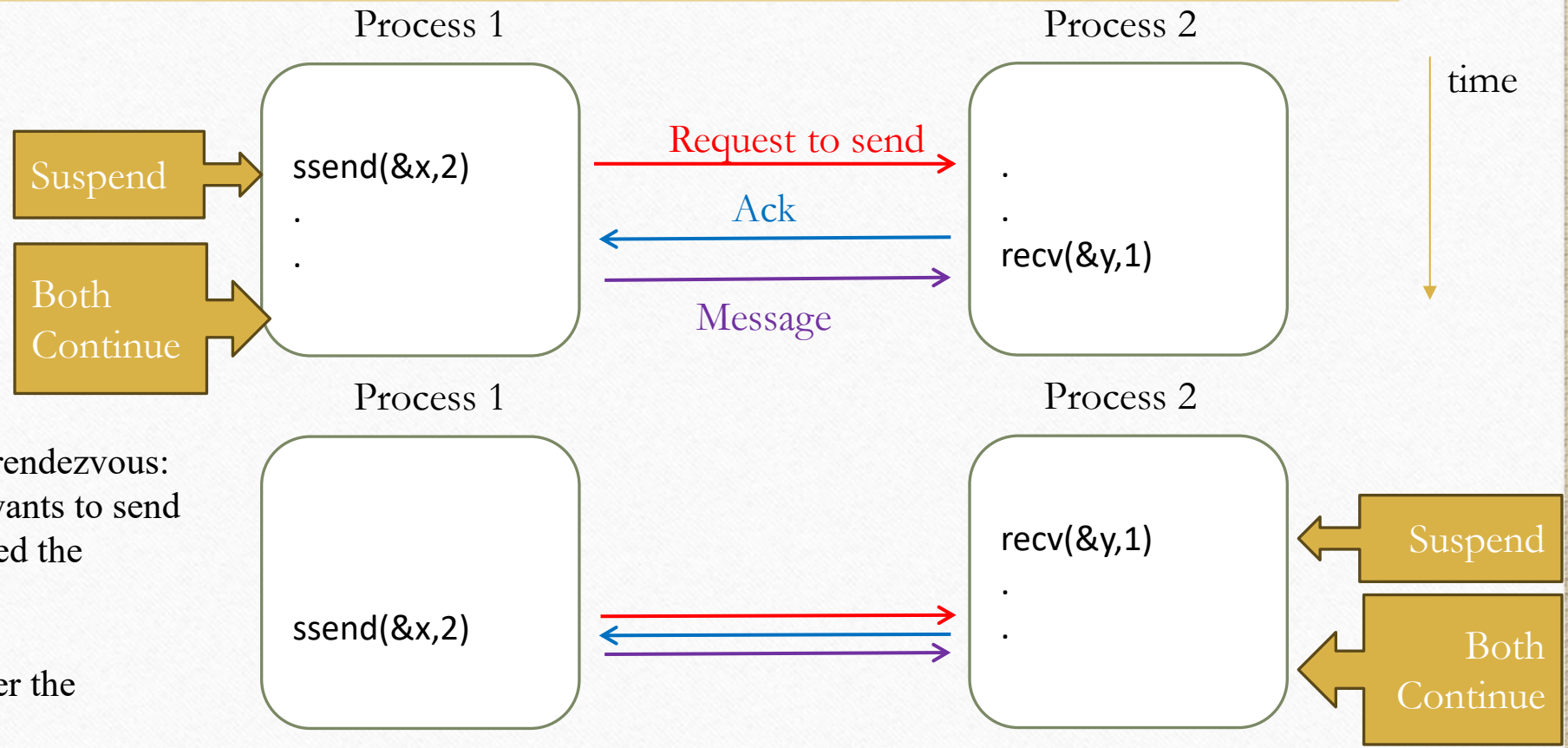
`MPI_Ssend` is blocking + synchronous
`MPI_Issend` is non-blocking + synchronous
`MPI_Send` blocking standard send
`MPI_Recv` non-blocking standard receive

Synchronous send: rendezvous semantics

- Let's consider a logical synchronous `ssend` primitive
 - It returns when the message transfer is completed
- `ssend` has rendezvous semantics:
 - The send cannot complete until the matching receive has started
 - The buffer can be safely reused only when it returns
- In MPI, `ssend` is implemented by `MPI_Ssend`
- `ssend` performs two actions: transfers data **and** synchronizes processes



A possible rendezvous protocol for send

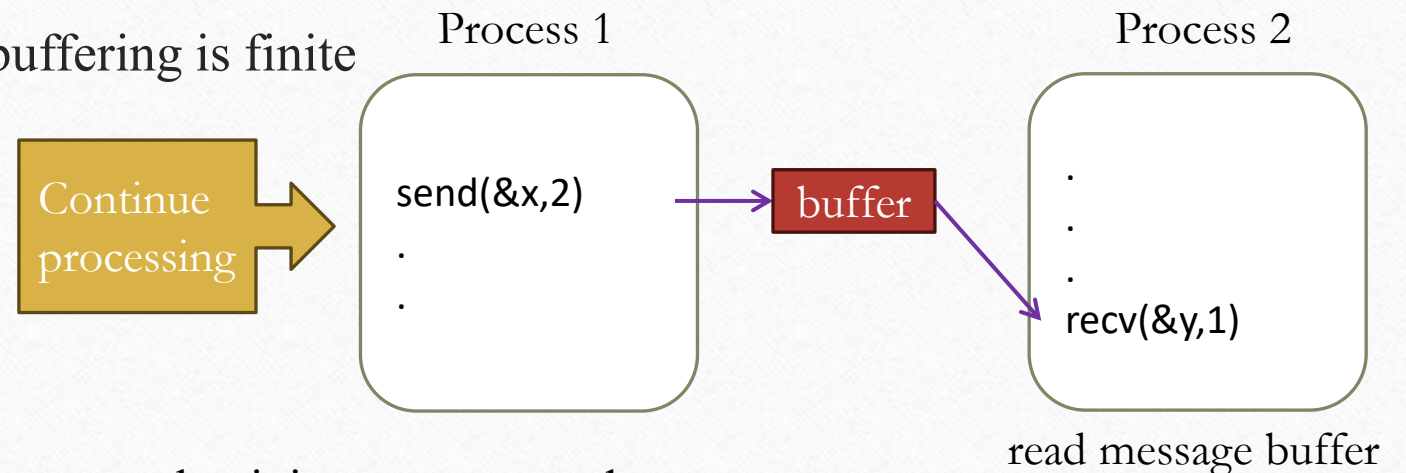


A possible implementation of rendezvous:

1. Sender announces that it wants to send
2. Receiver posts or has posted the matching receive
3. Data transfer can proceed
4. Sender completes only after the receiver participates

Asynchronous send: buffered behavior

- The sender may complete without waiting for the receiver to start the matching receive
- This usually requires buffering, e.g., copying data into an MPI internal buffer
- There is no rendezvous synchronization with the receiver
- The sender can continue, but internal buffering is finite



`MPI_Send` may behave this way for small messages, but it is not guaranteed

For large messages, `MPI_Send` may block until the matching `MPI_Recv` is posted (i.e., rendezvous).

This is still compliant with the blocking send semantics: when it returns the send buffer can be safely reused

MPI definitions of Blocking and Non-blocking

- **Blocking:** A blocking call returns after the **local part** of operation is completed
 - For a blocking send, the send buffer can be safely reused when the call returns
 - This does not imply that the matching receive has completed
 - With limited buffer capacity, a blocking send may behave as a synchronous send
- **Non-blocking:** A non-blocking call **returns immediately** after the operation is started
 - The operation remains pending and may progress while the program continues
 - **The user buffer involved in a non-blocking operation must not be accessed unsafely before completion**
 - Do not modify (or free) the user buffer until you know the operation has completed!
 - It is left to the programmer to ensure this (e.g., by using `MPI_Wait/MPI_Test` or their variants)
 - Non-blocking calls let you **overlap communication and computation**

Blocking communication and deadlock

- Example:

```
if (myrank == 0) {  
    MPI_Send(buf, count, MPI_INT, 1, 100, MPI_COMM_WORLD);  
    MPI_Recv(buf, count, MPI_INT, 1, 100, MPI_COMM_WORLD, MPI_STATUS_IGNORE);  
} else if (myrank == 1) {  
    MPI_Send(buf, count, MPI_INT, 0, 100, MPI_COMM_WORLD);  
    MPI_Recv(buf, count, MPI_INT, 0, 100, MPI_COMM_WORLD, MPI_STATUS_IGNORE);  
}
```

The `MPI_Send` may block until the matching receive is posted or enough buffering is available. If it blocks, then deadlock!

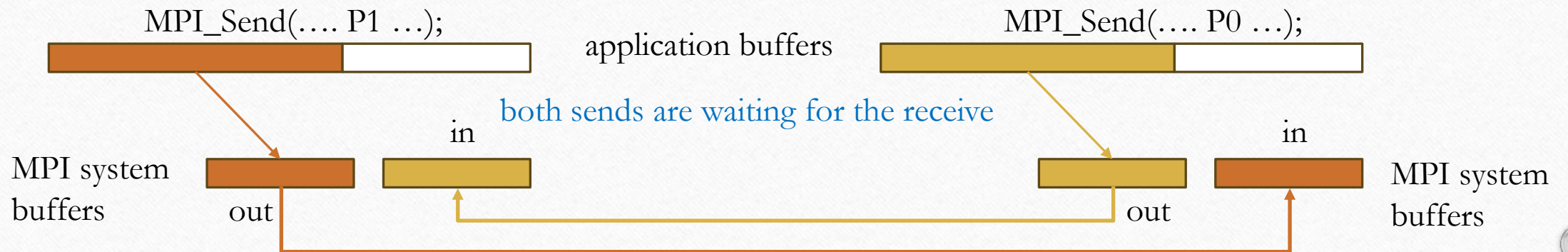
- Another case: sending data to the right-hand side neighbor on a ring

- The same problem as Dijkstra's "Dining Philosopher problem"
- Deadlock can be easily removed by breaking the circular wait

```
if (myrank%2 == 1) {  
    MPI_Send(...); MPI_Recv(...);  
} else {  
    MPI_Recv(...); MPI_Send(...);  
}
```


Why can MPI_Send block?

- MPI_Send returns when it is safe to reuse the application buffer
- This may happen because
 - the message has been copied into an MPI/system buffer, or
 - the matching receive has been posted and data transfer can proceed.
- The MPI standard permits the use of a system buffer but does not necessarily require it
- For large messages, MPI_Send may use a rendezvous protocol



Non-blocking send/receive

- `int MPI_Isend(const void* buf, int count, MPI_Datatype dt, int dest, int tag, MPI_Comm comm, MPI_Request* req)`
- `int MPI_Irecv (void* buf, int count, MPI_Datatype dt, int source, int tag, MPI_Comm comm, MPI_Request* req)`
- Processing continues immediately without waiting for the completion of the operation
- A communication request handle (`req`) is returned and must be used to test or wait for completion
- By calling `MPI_Wait()` or `MPI_Test()` (or their `*all` variants), it is possible to determine when the operation completes. Only then is it safe to reuse (or free) the message buffer
- It is okay to also mix non-blocking and blocking calls
 - `MPI_Isend()` ↔ `MPI_Recv()`, and, `MPI_Send()` ↔ `MPI_Irecv()`

Besides avoiding deadlocks, non-blocking communication is usually employed **to overlap computation and communication**

Waiting and checking non-blocking calls

- `int MPI_Wait(MPI_Request* req, MPI_Status* status)`
 - Blocks until a non-blocking send or receive operation has completed
 - After the `MPI_Wait` returns, the request becomes `MPI_REQUEST_NULL`
 - Use `MPI_STATUS_IGNORE` if status information is not needed
 - For waiting multiple non-blocking operations: `MPI_Waitany()`, `MPI_Waitall()`, `MPI_Waitsome()`
- `int MPI_Test(MPI_Request* req, int* flag, MPI_Status* status)`
 - Tests whether a non-blocking send or receive has completed
 - `flag` is set to 1 if the operation has completed, 0 otherwise
 - If `flag` = 1, the operation has completed and the request becomes `MPI_REQUEST_NULL`
 - For checking multiple non-blocking operations: `MPI_Testany()`, `MPI_Testall()`, `MPI_Testsome()`

MPI non-overtaking property: matching order

- If a single-threaded sender sends two messages using the same destination, tag and communicator, and the receiver posts two `MPI_Irecv` with the same source, communicator, and tag:
 - msg1 matches req1
 - msg2 matches req2
 - msg2 cannot overtake msg1 in message matching
- However, completion ordering is not guaranteed
- With `MPI_Waitany`, req2 may complete before req1
 - Message matching order is guaranteed
 - Request completion order is not guaranteed

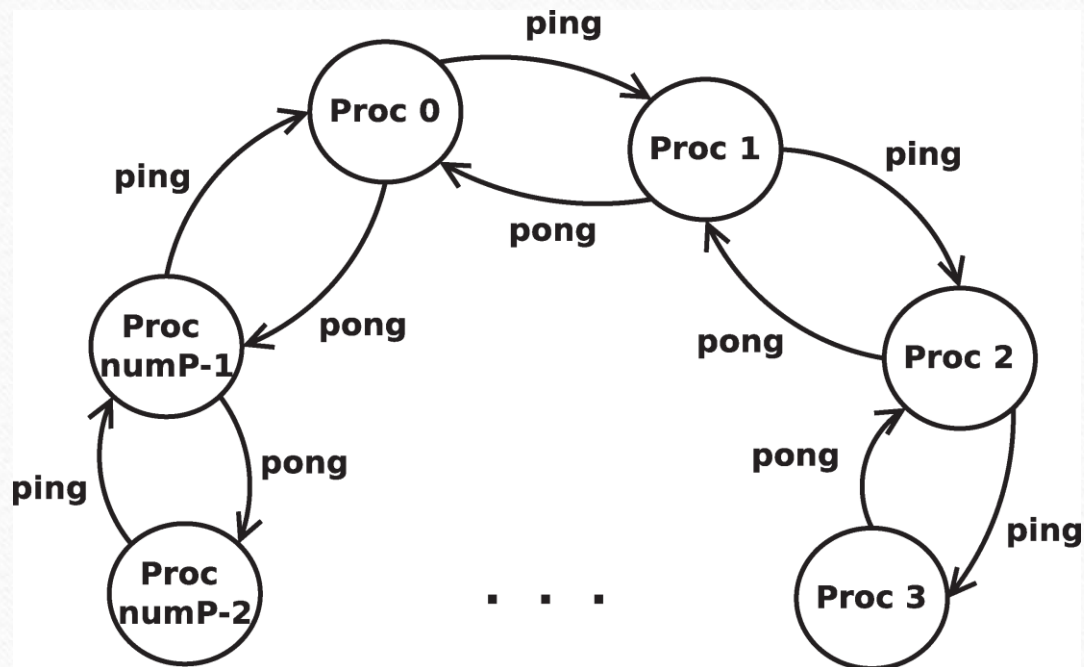
```
// sender side
MPI_Send(msg1, ... , dest, TAG, comm);
MPI_Send(msg2, ... , dest, TAG, comm);

// receiver side
MPI_Irecv(buf1, ... , source, TAG, comm, &req1);
MPI_Irecv(buf2, ... , source, TAG, comm, &req2);
```


Message exchange

- `int MPI_Sendrecv(const void* sndbuf, int sndcount, MPI_Datatype snddt, int dest, int sndtag, void* rcvbuf, int rcvcount, MPI_Datatype rcvdt, int source, int rcvtag, MPI_Comm comm, MPI_Status* status)`
 - `MPI_Sendrecv` is blocking: it returns when both the send and the receive parts have completed locally
 - Why `Sendrecv`? To avoid deadlock. MPI internally schedules the send and receive parts so common pairwise exchange patterns avoid deadlock
 - Useful to exchange data with a neighbor, e.g., in rings, stencils, and halo exchanges
 - `MPI_Sendrecv_replace` uses the same buffer for both send and receive

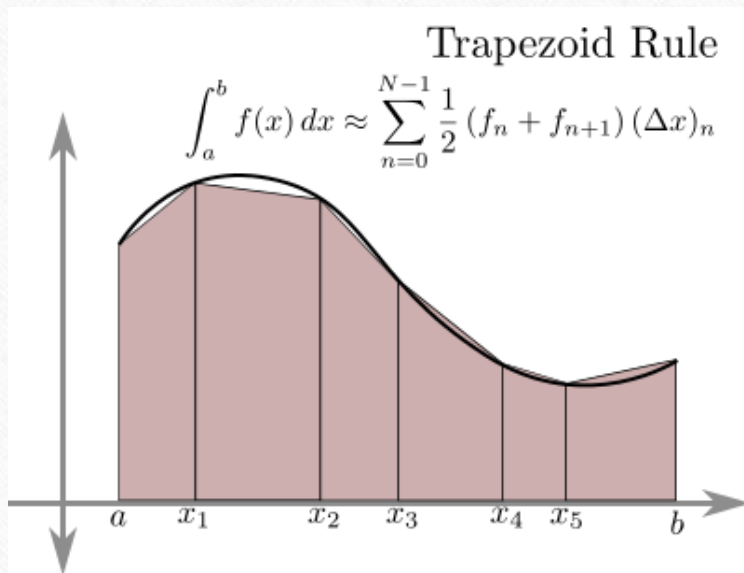
Example: ping-pong on an ordered ring



Pattern: each process sends PING clockwise and PONG counter-clockwise using non-blocking point-to-point communication

```
[torquati@spmln spmcode10]$ srun make mpi_pingpong_nb
mpicxx -std=c++20 -I. -I./include -Wall -O3 -march=native -ffast-math -o mpi_pingpong_nb
[torquati@spmln spmcode10]$ srun --mpi=pmix -N 8 --tasks-per-node=1 ./mpi_pingpong_nb 10
Process 4 posts PING number 1 to process 5
Process 7 posts PING number 1 to process 0
Process 5 posts PING number 1 to process 6
Process 6 posts PING number 1 to process 7
Process 0 posts PING number 1 to process 1
Process 1 posts PING number 1 to process 2
Process 3 posts PING number 1 to process 4
Process 2 posts PING number 1 to process 3
Process 0 receives PING number 1 from process 7
Process 0 posts PONG number 1 to process 7
Process 5 receives PING number 1 from process 4
Process 5 posts PONG number 1 to process 4
Process 1 receives PING number 1 from process 0
Process 6 receives PING number 1 from process 5
Process 4 receives PING number 1 from process 3
Process 1 posts PONG number 1 to process 0
Process 4 posts PONG number 1 to process 3
Process 4 receives PONG number 1 from process 5
Process 0 receives PONG number 1 from process 1
Process 0 posts PING number 2 to process 1
Process 5 receives PONG number 1 from process 6
Process 6 posts PONG number 1 to process 5
Process 4 posts PING number 2 to process 5
Process 5 posts PING number 2 to process 6
Process 7 receives PING number 1 from process 6
Process 7 posts PONG number 1 to process 6
```


Example: numerical integration

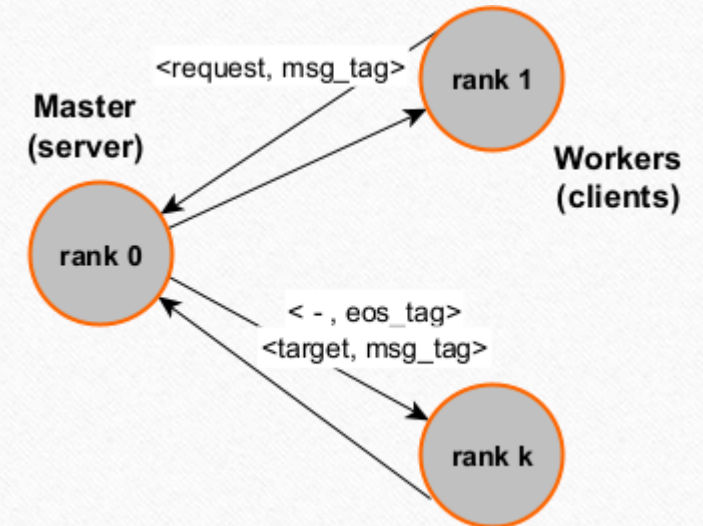


```
/*  
 * Compute the area of function f(x) for x=[a, b].  
 * The interval is partitioned into n subintervals of equal size.  
 */  
double trap(double a, double b, int n) {  
    double area = 0.0;  
    const double h = (b-a)/n;  
    double x = a;  
  
    for (int i = 0; i<n; ++i) {  
        area += h*(f(x) + f(x+h))/2.0;  
        x += h;  
    }  
    return area;  
}
```

MPI parallelization: Split the interval into P sub-intervals, where P is the number of processes used. Each process computes the area of the local interval and sends the partial results to the master (process with rank 0), which computes the final sum. Simple [Map-Reduce pattern](#): local computation + manual reduction on rank 0

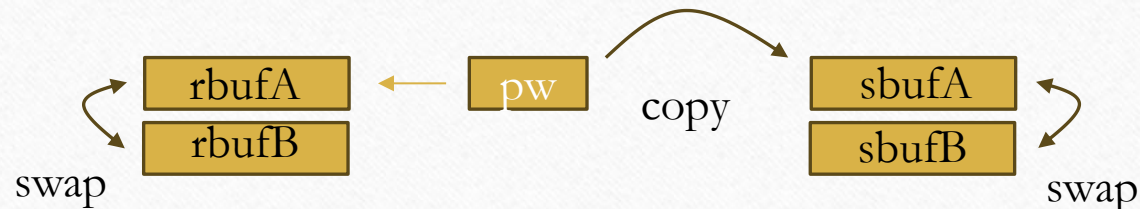
Example: Master-Workers

- Clients (Workers) send a random number of requests with TAG `msg_tag` to the Server and then a final message with TAG `eos_tag` (and zero-length payload) to signal the End-Of-Stream (EOS)
- The Server (rank 0) accepts requests from `MPI_ANY_SOURCE` and with `MPI_ANY_TAG` until all Workers send the EOS message
- The server uses `status.MPI_SOURCE` and `status.MPI_TAG` to identify the sender and the message kind
- The server terminates after receiving one EOS from each worker



Double buffering

- Technique used to hide communication latency and increase throughput
 - Used mainly for small/medium-sized data since it requires more memory
- It uses two communication buffers per neighbor peer (e.g., bufA, bufB)
 - While bufA is «in flight», bufB can be used to compute
 - On completion, swap buffers, so communication of the next round overlaps computation of the current round



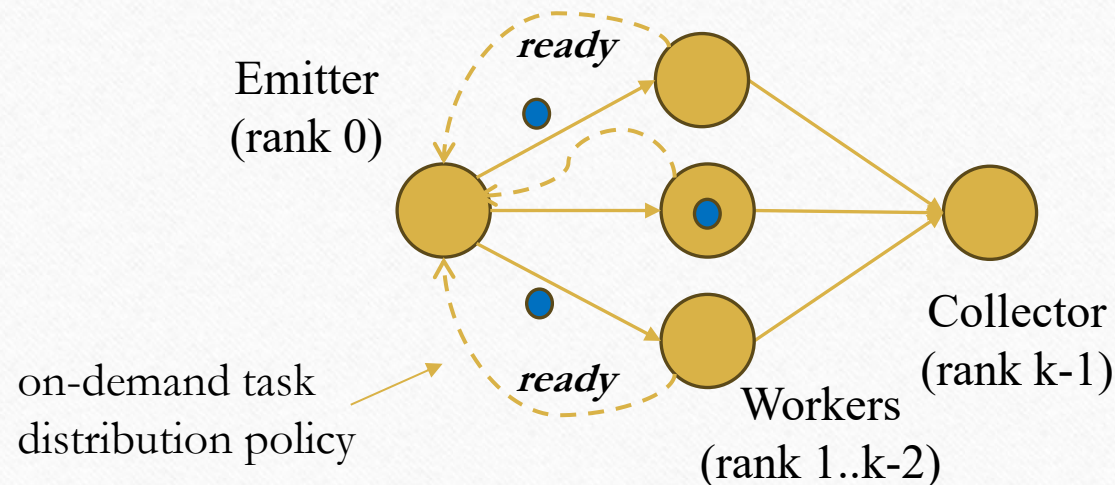
pseudo-code

```
void *rbufA, *rbufB;    // receive buffers
MPI_Request *rA, *rB;   // receive requests
void *sbufA, *sbufB;    // send buffers
MPI_Request *sA, *sB;   // send requests

MPI_Irecv(...rbufA, rA );
MPI_Irecv(...rbufB), rB;
while(true) {
    MPI_Wait(rA,...); // wait for recv completion
    pw = rbufA;      // current work buffer
    swap({rbufA,rA}, {rbufB, rB}); // swap pointers
    MPI_Irecv( ...rbufA, rA); // re-post a receive
    do_work(pw);      // work on the received data
    MPI_Wait(sA, ...); // wait for send completion
    copy(pw, sbufA);
    MPI_Isend(...,sbufA, sA); // send data
    swap({sbufA,sA}, {sbufB, sB}); // swap pointers
}
```

Farm skeleton in MPI

- In this example, we implement a classical (standalone) **Farm** skeleton (Emitter-Workers-Collector) employing **asynchronous communications** and **double buffering** in all stages to maximize the overlap of computation and communication.
- Possible extensions: within a Worker, we can use OpenMP/CUDA to reduce Worker's service time
 - **NOTE:** Use `MPI_Init_thread` instead of `MPI_Init` if multiple threads call MPI routines



Suggested Readings

- Chapter 9 of the «Parallel Programming Concepts and Practice» course book
- MPI: A Message-Passing Interface Standard Version 4.1
 - <https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report.pdf>
- Open
- MPI documentation:
 - <https://docs.open-mpi.org/en/v5.0.x/>
- MPI tutorial
 - <https://hpc-tutorials.llnl.gov/mpi/>
- OSU micro-benchmarks:
 - <https://mvapich.cse.ohio-state.edu/benchmarks/>
 - C benchmarks installed in /opt/ohpc/pub/mpi/osu-7.4/ on the spmcluster